Department of Electrical and Computer Engineering
University of Thessaly

ECE333 - Digital Systems Laboratory

Christos Karagiannis 3544

October 28, 2024

Laboratory Assignment 1st

7-Segment Display Driver

Contents

1 Abstract **3**

2 Introduction **3**

3 Part A: 7-Segment Decoder **4**
3.1 Implementation 4
3.2 Verification 4
3.3 Experiment/Final Implementation 4

4 Part B: Driving Four Digits **5**
4.1 Implementation 5
4.2 Verification 6
4.3 Experiment/Final Implementation 6
4.4 Optional Part: Advanced Use of the MMCM Module 7
4.4.1 Implementation 7
4.4.2 Verification 7
4.4.3 Experiment/Final Implementation 7

5 Part C: Step-by-step Message Rotation using a Button **8**
5.1 Implementation 8
5.2 Verification 8
5.3 Experiment/Final Implementation 9

6 Part D: Step-by-step Message Rotation with Fixed Delay **9**
6.1 Implementation 9
6.2 Verification 10
6.3 Experiment/Final Implementation 10

7 Conclusion **10**

1 Abstract

This document outlines the programming process for the FPGA NEXYS A7-100T board, specifically detailing the circuit description of a 4-digit, 7-segment display driver. It covers the abstract design of the implemented modules, verification through simulation, as well as the debugging and correction of flawed initial assumptions.

2 Introduction

The objectives of this assignment are the implementation of: initially, a 4-bit character to 8-bit signal decoder (Part A), a driver for four 7-segment displays where messages will be shown (Part B), and the manual rotation of the message using a button (Part C) or automated rotation with a fixed delay (Part D). All of the aforementioned goals were successfully achieved, including the optional section of Part B.

3 Part A: 7-Segment Decoder

3.1 Implementation

In this section, we were tasked with implementing a decoder that takes a 4-bit input signal (representing the desired character) and outputs an 8-bit signal (we also control the decimal point via the decoder), with 0 representing an activated display segment and 1 representing a deactivated one. The output signal consists of 8 wires in the following order: a (MSB), b, c, d, e, f, g, dp (LSB). Furthermore, to describe the decoder, an `always` block was used with the input character in its sensitivity list, alongside a `case` block that drives the appropriate output based on the input.

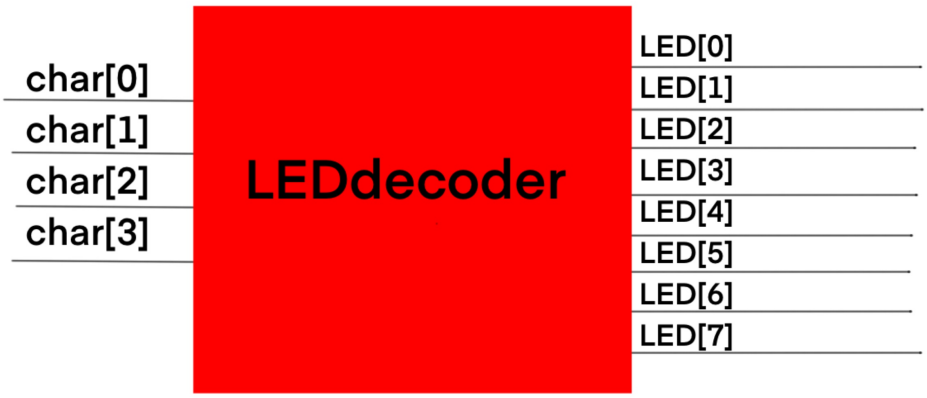

Figure 1: Dataflow of Part A.

3.2 Verification

To verify the circuit's behavior, we wrote a testbench providing every possible input value to confirm that the decoder produces the correct output.

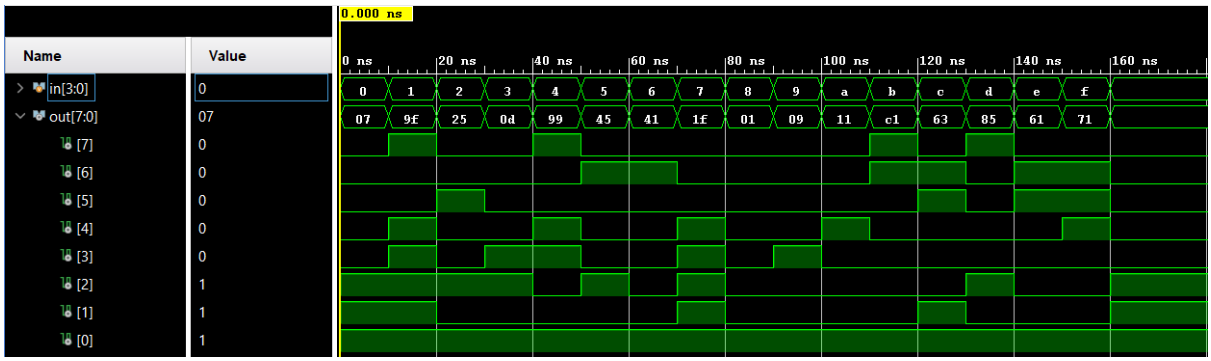

Figure 2: Waveforms of Part A.

3.3 Experiment/Final Implementation

Although programming the FPGA was not required for this specific part of the assignment, we verified that Vivado could successfully perform Synthesis and Implementation of the above

module.

4 Part B: Driving Four Digits

4.1 Implementation

The next step for the assignment involves describing a driver for the four 7-segment displays. However, to achieve the properly timed driving of the anodes and segments, an MMCM module is required to step down the board's clock from 100MHz to 5MHz. According to the board's manual, the equations to determine the appropriate parameter values for slowing down the initial clock are:

$$F_{VCO} = F_{CLKIN} \cdot \frac{M}{D} \qquad F_{CLKOUT} = F_{CLKIN} \cdot \frac{M}{D \cdot O}$$

where M, D, and O correspond to the parameters `.CLKFBOUT_MULT_F`, `.DIVCLK_DIVIDE`, and `.CLKOUT0_DIVIDE_F`, respectively. Furthermore, for the MMCM module, short-circuiting `CLKFBOUT` with `CLKFBIN` is essential.

Next, we describe the four-anode driver, which implements a Finite State Machine (FSM) using a 4-bit counter inside a `case` block. This setup serves to activate (setting the signal to 0) and deactivate (setting the signal to 1) each anode, switch the character displayed on each anode, and prepare the character to be displayed (since the board's interconnects exhibit resistance and capacitance, it is necessary to charge the cathodes two clock cycles before activating the anode). Finally, we use an `always` block sensitive to the positive edge of the clock (as this is a sequential circuit) and the positive edge of the reset signal to initialize and synchronize the circuit.

It is also necessary to implement three additional modules for the safe recognition of the reset signal, as it is triggered by the user pressing button N17. Specifically, we describe a D Flip-Flop module with an `always` block that sets the output to the input on the positive clock edge. We use three D Flip-Flops to create a synchronizer, which takes the asynchronous signal, passes it through the 3 D Flip-Flops, and outputs the signal of the 2nd and 3rd (thus eliminating potential metastability at the output of the 1st). These signals are fed into a debounce module, as there is a possibility that the button's internal spring may oscillate and produce false pulses either when pressed or released. To prevent this, we compare the two inputs and use a counter to introduce the necessary delay. The counter increments using an `always` block sensitive to the positive clock edge. Once we reach the desired value (equivalently, the desired time interval has passed) and both signals have the same value, we can forward the value to the debounce output; otherwise, we reset the counter and start over.

Finally, we use the decoder from Part A to appropriately interpret the character provided by the driver and activate/deactivate the cathode signals.

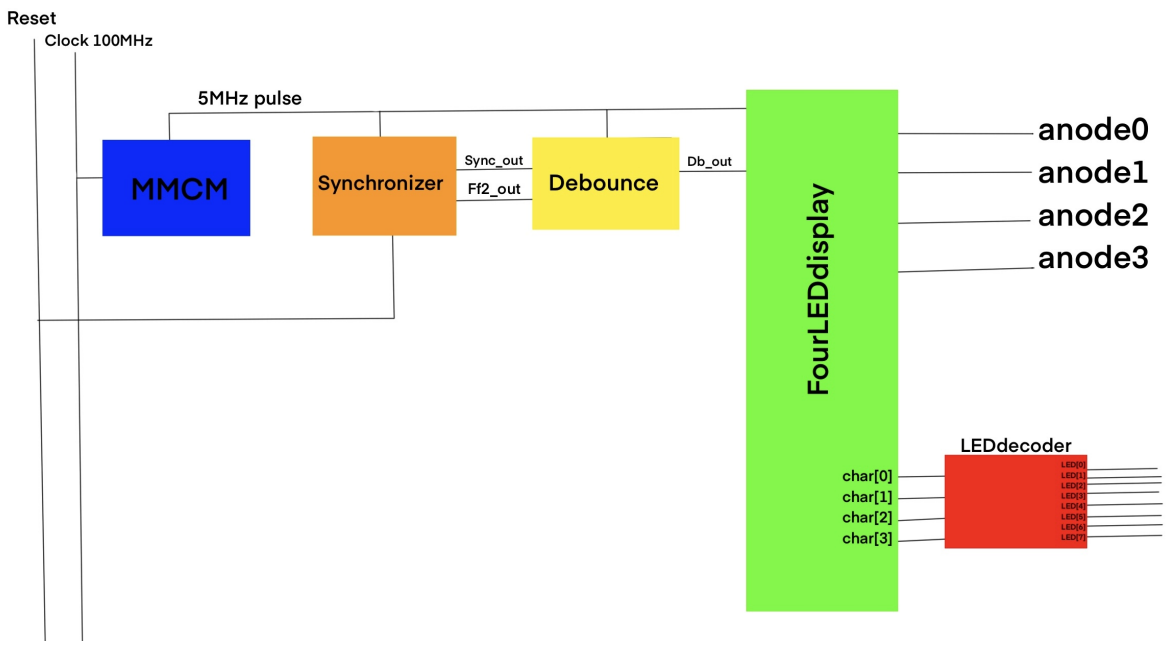

Figure 3: Dataflow of Part B.

4.2 Verification

To verify the circuit's behavior, we wrote a testbench where we initialized the clock, toggled it periodically every 5ns, and initialized the circuit with a pulse from the reset signal.

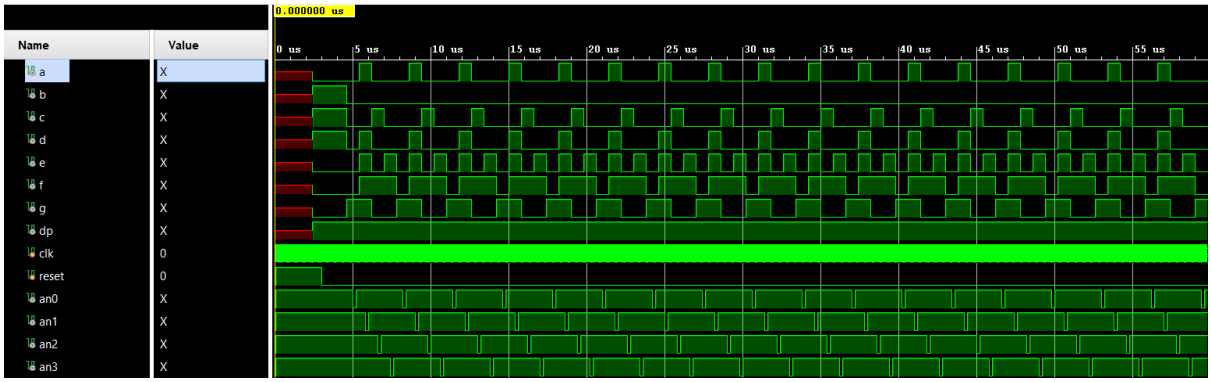

Figure 4: Waveforms of Part B.

4.3 Experiment/Final Implementation

The only issue encountered was that in the initial implementation, the displays flickered rapidly, and the LEDs lacked brightness. The root cause was that when the counter controlling the anodes reached the desired value, e.g., 1110, I switched to the next character on the immediate next increment, 1111, creating an overlap between the two characters.

The solution, therefore, was to delay the entire process by one cycle (equivalently, on the next counter value), resulting in stable and brightly lit characters on the displays.

4.4 Optional Part: Advanced Use of the MMCM Module

4.4.1 Implementation

For the optional step, the only necessary change is assigning new values to the MMCM module parameters. Specifically, to achieve further reduction of the clock frequency from 100MHz to 40KHz, we utilize the functionality provided by the `CLKOUT4_CASCADE` parameter, which routes `CLKOUT6` as an input to `CLKOUT4`, allowing us to re-divide the pulse using the `CLKOUT4_DIVIDE` parameter.

Using the same equations mentioned above, the parameter values are set as follows: `CLKFBOUT_MULT_F = 24`, `DIVCLK_DIVIDE = 4`, `CLKOUT6_DIVIDE = 120`, `CLKOUT4_DIVIDE = 125`, and `CLKOUT4_CASCADE = TRUE`. The Dataflow remains the same as in Figure 3.

4.4.2 Verification

We use the same testbench as before, resetting the circuit first and then displaying the message 3210.

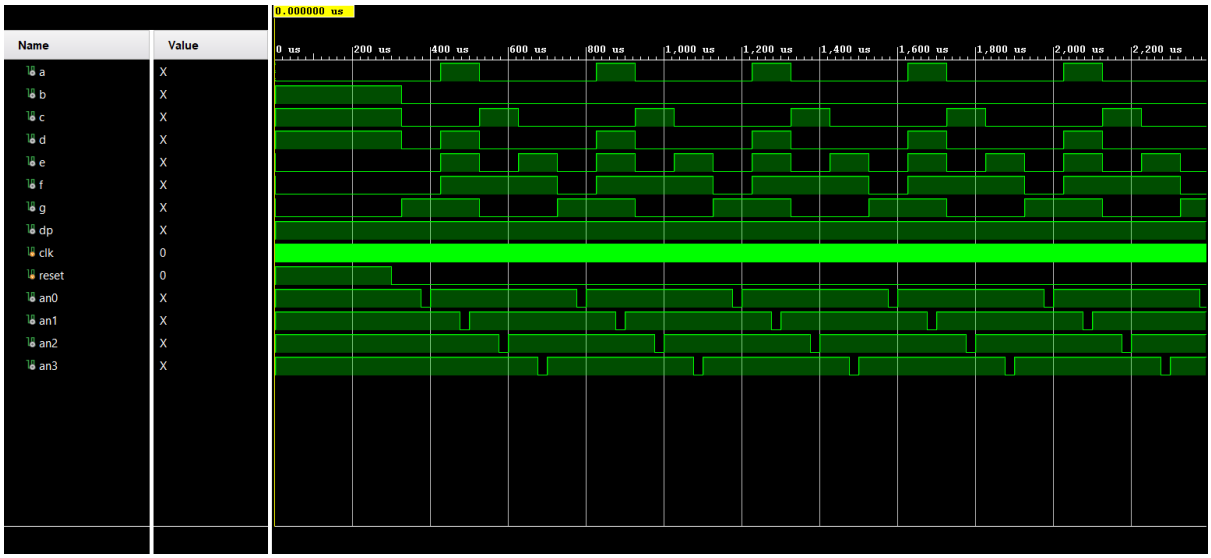

Figure 5: Waveforms of the Optional Part.

4.4.3 Experiment/Final Implementation

During board programming, the only noticeable change was that the displays were slightly dimmer, without any flickering. This is expected due to the longer time intervals during which the displays remain off.

5 Part C: Step-by-step Message Rotation using a Button

5.1 Implementation

Having properly driven the 4 displays, we can now introduce manual message rotation for the characters on the displays using a button.

First, we implement a `Memory` module, responsible for storing the display message, detecting the state of the button signal (when we press/release the button), and mapping characters to the correct displays. The module's inputs are the 5MHz clock, the reset signal, and the `btn` signal, which is synchronized and filtered since it is activated/deactivated by the user. Inside an `always` block sensitive to the positive edge of the clock and the positive edge of the reset signal, we check if the user has pressed button M17 (`btn` in the Verilog file). If so, we increment the `address` counter, which indicates which part of our memory is loaded: the value on display 2 shifts to display 1, 3 shifts to 2, 4 shifts to 3, and finally, we load a new value from memory into display 4. The chosen message to cycle through is `0123456789abcdef`.

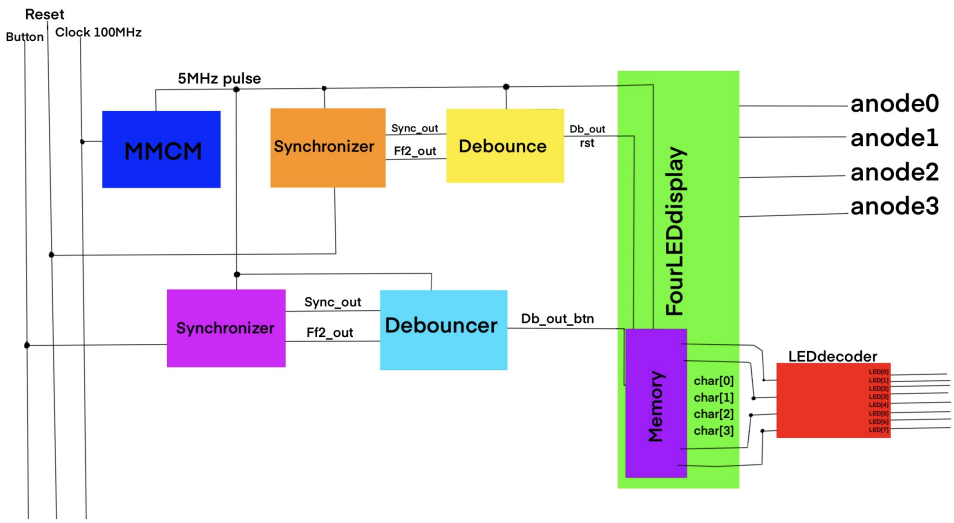

Figure 6: Dataflow of Part C.

5.2 Verification

To verify this step, we wrote a testbench to reset the circuit and generate a periodic pulse on the `btn` signal, ensuring the characters cycled correctly across all displays.

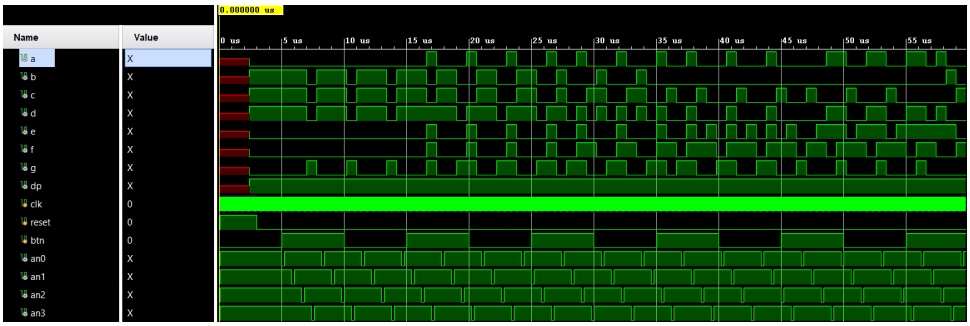

Figure 7: Waveforms of Part C.

5.3 Experiment/Final Implementation

During physical testing on the board, pressing the button shifted the characters by one position, but releasing it caused them to jump 2 or 3 additional positions. To fix this, I introduced a register inside the module to store the previous state of the `btn` signal (`btn_prv`). Now, the necessary condition for a 1-position rotation is that `btn` equals 1 and `btn_prv` equals 0. Otherwise, whether the button is held down or released, the message remains stable.

6 Part D: Step-by-step Message Rotation with Fixed Delay

6.1 Implementation

The final part of the assignment is a variation of Part C. The only difference is that instead of the `Memory` module receiving a `btn` signal from the user, it utilizes an internal 23-bit counter, where:

$$t = 2^{23} \cdot 200ns = 1.6777214 \text{ sec}$$

thus introducing the desired delay for shifting our message by one position. So, while the sensitivity list of the `always` block remains the same as in Part C, on every clock edge, instead of checking if the `btn` signal is 0 or 1, we increment `cnt` and compare it to the value `7FFFFF`. If they match, we reset `cnt` to zero, and the exact same character shifting process from Part C takes place.

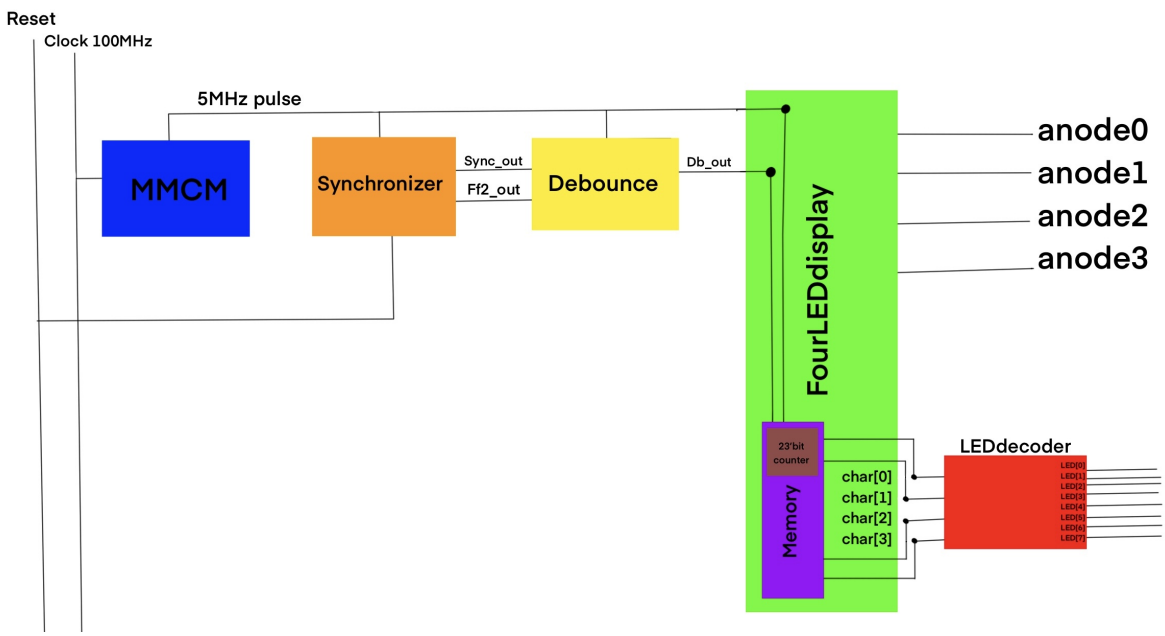

Figure 8: Dataflow of Part D.

6.2 Verification

To verify this part, we wrote a testbench where we simply reset the circuit and observed the waveforms automatically changing with a constant delay.

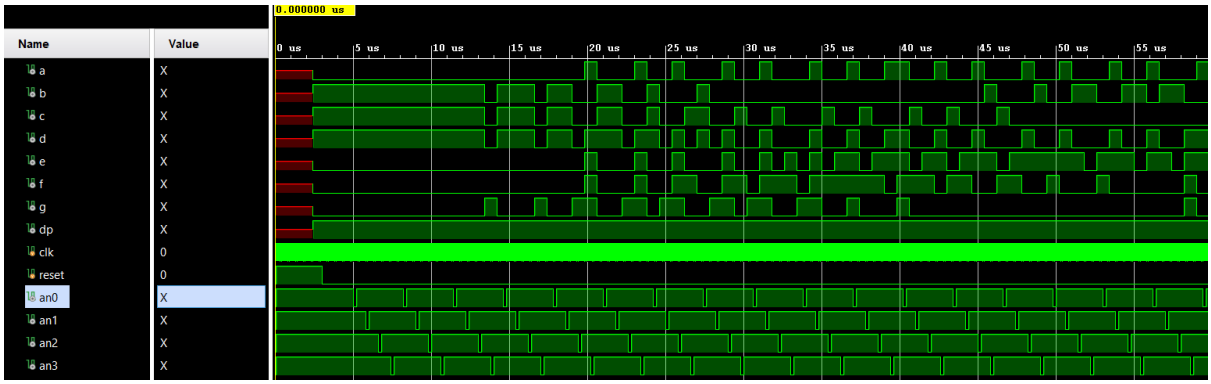

Figure 9: Waveforms of Part D.

6.3 Experiment/Final Implementation

No issues arose during board programming. The transition from Part C to Part D was smooth, and because the logic is quite similar, it worked on the first try.

7 Conclusion

This was an excellent first experience with designing, describing, and physically implementing circuits on the FPGA. Through this practical assignment, I began to understand the relationship between how I describe the circuit and the actual physical implementation, as well as what to expect as a final result from Vivado.